PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 1
2. Background & Business Objective 1
3. Product Purpose 2
4. System Functionalities 2
5. User Stories & Use Cases 3
6. Features Included (Scope Definition) 3
7. Features Not Included (Scope Control) 3
8. Assumptions & Constraints 4
9. Risks & Questions Throughout Development 4

Email: umhackathon@um.edu.my

Project Name: DuitOnTrack

Version: 1.0

1. Project Overview

- **Problem Statement:** Malaysian university students typically live on RM 400-800 per month from allowances, PTPTN, and part-time work. The problem is rarely one big purchase, instead they overspend on small ones, daily decisions; a RM 6 bubble tea, a RM 25 Grab ride, a midnight Shopee impulse buy. Traditional budgeting apps tell students what they already spent, after the damage is done. There is no tool that helps them decide in the moment, before they spend.

- **Target Domain:** Personal financial coaching for Malaysian university students for pre-purchase decision making.

- **Proposed Solution Summary:** DuitOnTrack has AI feature that gives quick decision for verdict, has interchangeable mode between 'Save' or 'Spend' mode. Generate report to see user's spending pattern. It has also recent decision strip to keep track of user's previous decision in their dashboard.

2. Background & Business Objective

- **Background of the Problem:** University life in Malaysia is financially tight by design, part-times money sometimes only cover the necessary purchase, allowance are only enough for bills and tuition fees. However, this constraint doesn't limit students to make rash decision to overspend on small purchase like milk drinks, coffees and impulsive online purchase. Existing budgeting tool only have the data once the student has spend their money on the items, what the student actually need is a tool that tells that to stop before the purchase is made, a decision-making tool.

Email: umhackathon@um.edu.my

- **Importance of Solving This Issue:** Financial habits that form during university years often sticks until adulthood. We aim to intervene this habits by giving the student direct decision-making that support saving habit so that they will keep on repeating this habit until they finish studying in university. A student that saves few bucks a month would always be better than student who did not even manage to save even a penny.

- **Strategic Fit / Impact:** DuitOnTrack aligns with Z.AI's mission of deploying accessible, practical AI for Southeast Asian users. The product uses ILMU-GLM-5.1 as its reasoning engine, deployed through the ilmu.ai MaaS platform, with Firebase as the backend for authentication, data persistence, and user memory. The system is designed to be lightweight, mobile-friendly, and operable entirely through a web browser with no app installation required. We hope that the student will have wiser spending decision, reach their savings goal and have more awareness on smarter financial habit.

3. Product Purpose

3.1. Main Goal of the System: To act as decision-making AI personal coach that helps Malaysian university student build smarter spending decision.

3.2. Intended Users (Target Audience):

- Malaysian university students (primary)
- Students on fixed monthly allowances or PTPTN
- Students with irregular side income (tutoring, part-time work)
- First-time budgeters with no prior financial management experience

4. System Functionalities

4.1. Description: DuitOnTrack operates as a context-aware AI coaching layer on top of a personal financial profile. The user sets up their budget once, then interacts with the coach conversationally. The system maintains memory of past decisions via Firebase, feeding recent history into every new AI prompt so the coach's advice improves over time and stays consistent with the user's actual financial situation.

4.2. Key Functionalities:

- **Coach AI:** The core feature. The user types any spending question in natural language and receives a verdict (Yes / Wait / No) with contextual reasoning that references their actual remaining budget, days left, and savings goals. Responses are delivered in Manglish to feel personal and relatable.
- **Mood Mode Toggle (Save/Spend):** Users can switch between two coaching personalities. Save Mode is strict, the coach pushes back on wants and protects savings. Spend Mode is relaxed, the coach acknowledges the user has earned some flexibility. The same question returns a different answer depending on the active mode, making the coach feel responsive to the user's life situation rather than mechanically rigid.
- **Savings Goals System:** Users can set multiple named savings goals (e.g. Langkawi Trip, New Laptop, Emergency Fund) with target amounts. Each goal tracks progress independently. The AI is aware of all active goals when generating advice, and references them explicitly in its reasoning.
- **Recent Decisions Strip:** The dashboard displays the user's previous coaching session whether that decided to follow or abandon the coach advice. The system lets user see their own decision-pattern over time.

- **Goal Report:** Each savings goal has its own report modal showing progress, saved amount vs target, that consists AI generated user pattern and next-month experiments.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: DuitOnTrack uses ILMU-GLM-5.1, provided by Z.AI through the ilmu.ai MaaS platform. This model was selected for three reasons: it is the designated model for UMHackathon 2026 participants; it supports the Anthropic Messages API format, enabling straightforward integration with our existing frontend architecture; and it demonstrates strong performance on conversational, context-rich prompts in mixed English-Malay (Manglish) which is essential for our target audience to feel that the coach speaks their language naturally as our primary target user is Malaysian university student.

4.3.2. **Prompting Strategy:** DuitOnTrack uses a multi-step contextual prompting strategy. Each AI call includes a structured system prompt that encodes the user's full financial context which is their budget remaining, days left, active goals, side income, and current mood mode, followed by a summary of the user's last previous 10 decisions pulled from Firebase as conversational memory. The user's question is then appended as the final user message. The strategy was chosen because generic financial advice is useless. The system prompt ends with a strict output instruction requiring the model to begin with VERDICT: YES, VERDICT: WAIT, or VERDICT: NO before any reasoning, ensuring reliable parsing regardless of response variation.

4.3.3. **Context & Input Handling**: The system prompt is constructed dynamically at runtime and typically runs 300–500 tokens. Chat history is capped at the 20 most recent exchanges to prevent context overflow. User questions are accepted as free-form text with no hard character limit enforced on the frontend, but the model's max_tokens is set to 1000 per response to keep replies concise and readable on mobile. Inputs that are very long or off-topic are

still processed but the model is prompted to redirect gracefully and ask the user to clarify with a specific item and amount.

4.3.4. **Fallback & Failure Behavior:** If the model returns a response without a parseable VERDICT: tag, the application defaults to displaying a Wait verdict, the most conservative outcome, which protects the user from spending without clear guidance. If the API call to /api/chat fails entirely due to a network error or timeout, the chat interface displays: *"Sorry, I couldn't process that. Try again!"* No crash or blank screen occurs. The server-side proxy handles all credential management, so no API keys are exposed in the browser at any point.

5. User Stories & Use Cases

- **User Stories:**
  - As a university student, I want to ask whether I can afford something right now, so that I don't overspend without realising it.
  - As a student with a savings goal, I want the coach to factor in my goal when giving advice, so that my spending decisions actively move me closer to what I am saving for.
  - As a student who just got paid from tutoring, I want to switch to Spend mode, so that I can enjoy spending a little without feeling guilty.

- **Use Cases (Main Interactions):**

Coach AI Flow: User types a spending question on the dashboard or Coach AI page → system builds a context-rich prompt with the user's financial profile and decision history → sends to /api/chat server-side proxy → proxy forwards to ILMU-GLM-5.1 → model returns verdict and reasoning → response displayed in chat interface → user taps Follow or Ignore → decision and chat entry saved to Firestore → dashboard decisions strip updates.

Goal Setup Flow: User clicks "+ Add goal" on dashboard → fills in goal name, target amount, current savings, and budget information → data saved to Firestore under the user's document → goal appears on dashboard with progress bar → AI is immediately aware of the new goal in all subsequent coaching sessions.

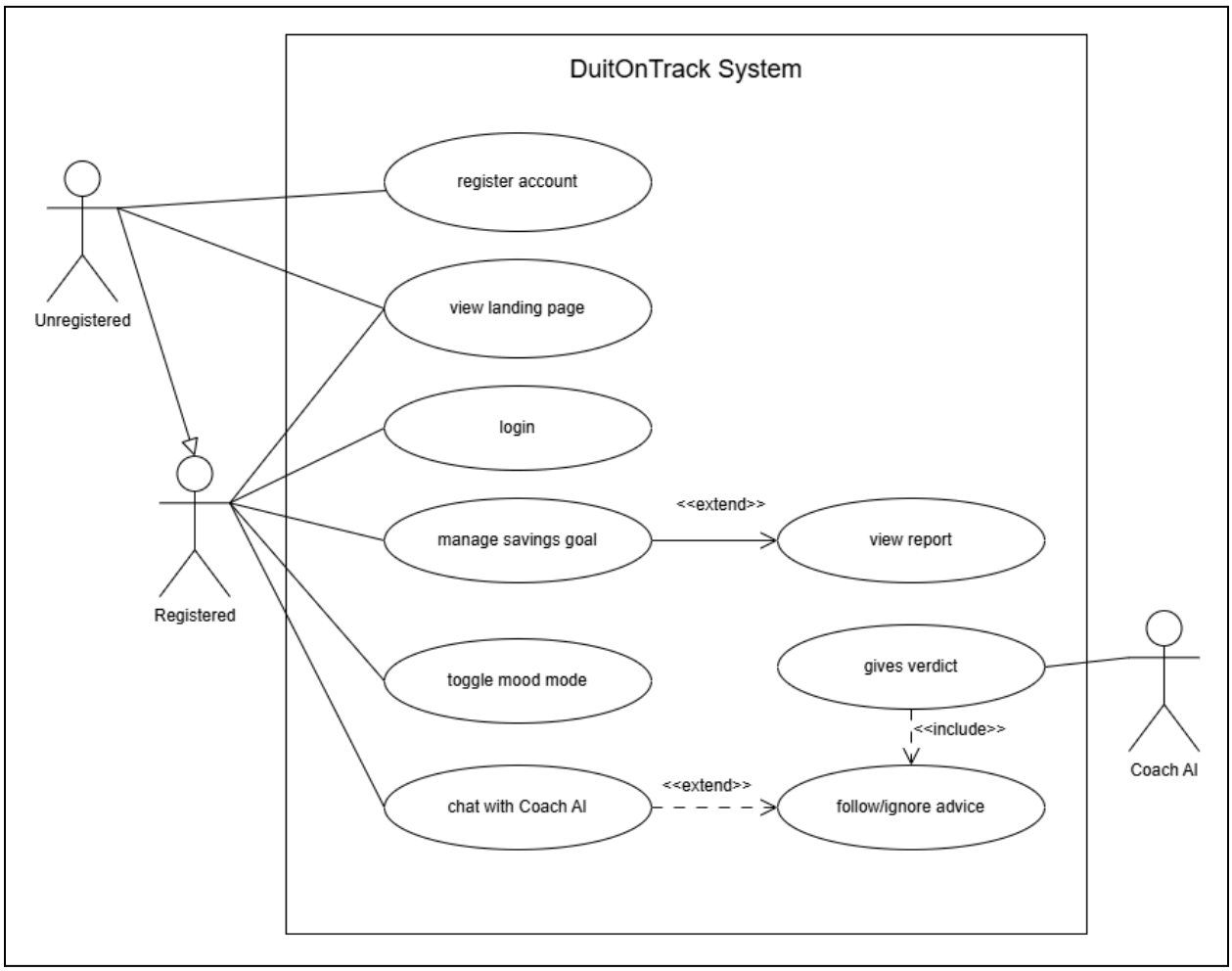
Goal Report Flow: User clicks Report on any goal card → modal opens showing saved amount, target, and percentage complete → any monthly reports stored in Firestore for that goal are displayed with pattern insights and suggested experiments → user can navigate directly to the Coach AI page from the modal.

Mode Toggle Flow: User taps Save or Spend on dashboard or chat sidebar → mode saved to Firestore → all subsequent AI responses in the session reflect the updated coaching personality → toast notification confirms the switch.

Auth Flow: New user clicks "Start Now" on landing page → registration modal appears → user registers via email/password or Google sign-in → Firebase Auth creates the account and user document in Firestore → user is redirected to dashboard. Returning user logs in → Firebase Auth verifies identity → Firestore loads profile, goals, decisions, and chat history → dashboard populates with the user's data.

Email: umhackathon@um.edu.my

- **Use Case Diagram:**



Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- AI-powered Coach AI spending verdict with contextual reasoning in Manglish.
- User authentication via email/password and Google sign-in (Firebase Auth).
- Persistent user profiles, goals, chat history, and decisions stored in Firestore.
- Multiple savings goals with independent progress tracking.
- Mood Mode toggle (Save /Spend) that changes coaching personality.
- Recent decisions strip on dashboard showing follow/ignore history
- Per-goal report modal with progress metrics and monthly AI summaries
- Server-side API proxy (/api/chat) for secure, credential-safe calls to ilmu.ai
- Fully responsive web app that works on mobile browsers without installation
- Public landing page accessible without login; all personal features gated behind authentication

7. Features Not Included (Scope Control)

- Automatic bank account or e-wallet integration (no read access to actual transactions).
- Push notifications or scheduled reminders.
- Peer or social features (no sharing, leaderboards, or group savings).
- Native mobile app (iOS or Android), web only for this version.
- Automated monthly report generation (report content requires manual trigger in current version).
- Multi-currency support, as for now DuitOnTrack only support Malaysian Ringgit (RM) only.
- Offline mode as active internet connection required for all AI features.

Email: umhackathon@um.edu.my

8. Assumptions & Constraints

- **LLM Cost Constraint:** An average user session is estimated at 3-5 AI interactions, each consuming approximately 600-800 tokens (system prompt plus history plus question plus response). This puts a typical session at roughly 3,000-4,000 tokens. To manage inference costs at scale, the system caps active prompt history at 10 messages, preventing runaway context growth. The max_tokens ceiling of 1,000 per response keeps output costs predictable. At hackathon-provided API access rates, cost per session is negligible.

- **Technical Constraints:** The app is a static web application HTML, CSS, and vanilla JavaScript is implemented with the Firebase SDK loaded via CDN. AI calls are routed through a server-side proxy endpoint (/api/chat) which handles credential management and forwards requests to the ilmu.ai API. Firebase Spark plan (free tier) is used, which imposes daily read/write limits sufficient for hackathon-scale usage.

- **Performance Constraints:** AI response time depends on ilmu.ai server load, typically 5-10 seconds per query. A typing indicator is shown during this wait to maintain perceived responsiveness. However during peak usage, ilmu.ai server would take longer time to load, which could exceed 1 minute per query. The app does not cache AI responses, as each answer is personalized to the user's current financial state and would be stale if reused.

Email: umhackathon@um.edu.my

9. Risks & Questions Throughout Development

- **Advice Accuracy Risk:** The AI coaches based on declared budget figures, not actual bank balances as the system is not yet integrated with the real banking system. A user who has already spent more than they reported will receive advice that is too permissive. Mitigation: the app is framed as a coaching tool, users are reminded that advice quality depends on the accuracy of their inputs, a fake input would give inaccurate advice.

- **Ignored Advice Loop:** A user who consistently ignores the coach's advice still receives the same style of responses. There is currently no escalation path where the coach becomes more forceful toward user after repeated ignored decisions. This could reduce long-term engagement. A future version could detect ignored patterns and adjust coach tone accordingly toward the user's manner.

- **Hallucination Risk:** The model may occasionally generate advice that references incorrect figures or invents context not present in the prompt. Mitigation: the system prompt is highly structured and data-specific, reducing the space for hallucination.

- **Scope Creep During Hackathon:** The team identified several desirable features (automated report generation, bank API integration, push notifications) that were explicitly designed to maintain a focused, functional core product within the hackathon timeline.

Email: umhackathon@um.edu.my